

ROSMASTER R2 Outdoor Racing

ROS Melodic · Ubuntu 18.04 · TG30 Tilted LiDAR

What This Code Does

This code lets your ROSMASTER R2 robot drive a sidewalk or driveway race course all by itself. You drive the course once with the joystick to teach it the path. Then it races on its own and avoids curbs and grass edges automatically. You can grab the joystick and take over at any time during racing.

The steps every time you want to race:

Step	What you do
1	Start the car, lidar, and IMU
2	Build a map with Cartographer
3	Save the map
4	Start localization so the robot knows where it is
5	Drive the course to record the race path
6	Close the path into a loop
7	Race

One-Time Setup

Open a terminal on the robot and run these commands once:

```
export ROBOT_TYPE=R2
echo "export ROBOT_TYPE=R2" >> ~/.bashrc
source ~/.bashrc
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

The Joystick

The joystick works all the time without pressing anything special. Moving the joystick always overrides the robot even while LB is held. Joystick priority is 90. Autonomous priority is 50. Higher number wins.

Control	What it does
Left stick up / down	Drive forward and backward
Right stick left / right	Steer left and right
LB button — hold down	Robot drives itself

LB button — let go	You take back control immediately
--------------------	-----------------------------------

How Steering Works (No Ackermann Bridge)

The R2 motor driver reads steering from `msg.linear.y` instead of the usual `msg.angular.z`. The joystick already sends `linear.y` the right way. Pure pursuit sends `angular.z`, so `twist_to_rosmaster.py` converts it automatically inside `racing_stack.launch`. You do not need to do anything extra.

Why Cartographer Instead of GMapping

Cartographer builds much better maps for racing. When you drive one full lap and come back to where you started, Cartographer matches the laser scan to the beginning of the map and snaps both ends together perfectly. This is called loop closure.

GMapping cannot do this. With GMapping the start and end of the map are often 30 cm to 1 metre apart. That makes the robot confused when it reaches the start line during racing. Always use Cartographer for this project.

Step 1 — Start the Car and LiDAR

Open Terminal 1. Keep it open the whole time.

```
roslaunch yahboomcar_nav laser_bringup.launch
```

This single command starts everything the robot needs:

- TG30 YDLidar — publishes laser scan on `/scan`
- Hardware driver — controls wheels and motors
- IMU Madgwick filter — smooths raw IMU into `/imu/imu_data`
- EKF — combines wheel odometry and IMU into smooth `/odom`
- URDF and TF tree — tells ROS the robot's shape and sensor positions
- Joystick — you can drive right now
- Scan pipeline — converts raw scan into `/scan_obstacles` (curb/grass edges included)

Check it is working — open Terminal 2 and test each topic:

```
rostopic hz /scan          # Good: shows ~7.000 Hz
rostopic hz /imu/imu_data  # Good: shows ~50.000 Hz
rostopic hz /odom          # Good: shows ~20.000 Hz
```

Press Ctrl-C after each check. Drive with the joystick to confirm the wheels move before going to Step 2.

Step 2 — Build the Map

Open Terminal 3:

```
roslaunch r2_tg30_race mapping_cartographer.launch
```

Open Terminal 4 to watch the map:

```
roslaunch r2_tg30_race view_mapping.launch
```

A window opens showing a gray map. Drive the robot and the map grows. Drive the full course slowly — about 0.2 to 0.3 metres per second. Complete at least one full lap.

What Loop Closure Looks Like

When you return to where you started, Cartographer automatically closes the loop. In RViz the map will shift slightly and then snap together cleanly. Wait about 10 seconds after you return to the start for this to settle.

You know the loop closed when:

- The map looks clean — no blurry edges or doubled lines
 - The path line in RViz makes a smooth complete circle
-

Step 3 — Save the Map

After loop closure, open Terminal 5 and run these three commands one at a time:

```
roslaunch map_server map_saver -f \
  $(rospack find r2_tg30_race)/maps/driveway_course
rosservice call /finish_trajectory 0
rosservice call /write_state \
  "{filename: '$(rospack find r2_tg30_race)/maps/driveway_course.pbstream'}"
```

Three files are now saved in the maps/ folder:

- driveway_course.pgm — the map picture
- driveway_course.yaml — map settings
- driveway_course.pbstream — Cartographer internal data

Press Ctrl-C in Terminal 3 to stop Cartographer. Press Ctrl-C in Terminal 4 to close RViz.

Step 4 — Start Localization

Terminal 1 must still be running. Open Terminal 3:

```
roslaunch r2_tg30_race localization_amcl.launch
```

Open Terminal 4:

```
roslaunch r2_tg30_race view_localization.launch
```

The saved map appears in RViz. You will see a red arrow (robot location) and a cloud of red dots (particle cloud).

Tell the Robot Where It Is

AMCL does not know its starting position. You must tell it:

- In RViz, click 2D Pose Estimate in the toolbar at the top
- Click on the map where the robot actually is right now
- Hold the mouse and drag in the direction the robot is facing
- Release the mouse

The red dots should gather into a tight cluster where you clicked. Do not go to Step 5 until the particle cloud is a tight cluster.

Step 5 — Record the Race Path

With Terminals 1 and 3 still running, open Terminal 5:

```
roslaunch r2_tg30_race record_waypoints_live.py \
  _spacing_m:=0.20 \
  _out_yaml:=$(rospack find r2_tg30_race)/waypoints/waypoints.yaml
```

In RViz a cyan line grows behind the robot as you drive. Each dot on the line is one waypoint saved every 20 centimetres. Drive the course at the speed you want the robot to race. When you finish your lap, press Ctrl-C in Terminal 5. The file saves automatically.

Check the File Saved Correctly

```
python3 -c "
import yaml
with open('$(rospack find r2_tg30_race)/waypoints/waypoints.yaml') as f:
    d = yaml.safe_load(f)
    print('frame_id:', d['frame_id'])
    print('waypoints recorded:', len(d['waypoints']))
"
```

You need to see `frame_id: map (not odom)` and `waypoints recorded: a number bigger than 50`. If it says `frame_id: odom`, localization was not running — redo Steps 4 and 5.

Step 6 — Close the Path into a Loop

Where you stopped the recording is not exactly where you started. This step fills in the gap with smooth bridge waypoints so the robot can race lap after lap without jumping.

```
python3 $(rospack find r2_tg30_race)/scripts/close_loop.py \
  --in_yaml $(rospack find r2_tg30_race)/waypoints/waypoints.yaml \
  --out_yaml $(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml \
  --spacing_m 0.20 \
  --close_thresh 2.0
```

You should see: `Loop closure complete`. If you get a gap error, either drive the course again and stop closer to the start, or change `--close_thresh 2.0` to `--close_thresh 4.0`.

Step 6b — Preview the Path in RViz

With Terminals 1 and 3 running, open Terminal 5:

```
roslaunch r2_tg30_race waypoint_map_builder.py \
  _yaml_file:=$(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml
```

Check these before racing:

- Yellow line is smooth with no big jumps or gaps
 - Yellow line makes a complete loop that closes on itself
 - Red arrow is roughly where the robot actually sits
-

Step 7 — Race

Terminals 1 (laser_bringup) and 3 (localization_amcl) must still be running.

Open Terminal 6:

```
roslaunch r2_tg30_race racing_stack.launch \
  waypoint_yaml:=$(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml
```

Open Terminal 7:

```
roslaunch r2_tg30_race view_racing.launch
```

To start autonomous racing: hold the LB button on the joystick. To stop: let go of LB — the robot stops right away. To take over manually: just move the joystick. You always win.

What You See in RViz During Racing

Display	What it means
Yellow line	The recorded race path
Orange ball	Where pure pursuit is aiming right now
Green dots	Curbs, grass edges, and other obstacles
Red arrow	Where the robot thinks it is
Red dot cloud	AMCL particles — tight cluster means confident
Green spheres	Each 20 cm waypoint on the path
Text AUTO or MANUAL	Whether the robot is driving itself

How Obstacle Avoidance Works

Two systems run at the same time. Pure pursuit steers the robot along the yellow race line. Follow the Gap (FTG) watches the laser for anything close and steers around it.

```
Nothing within 1.6 m      -&gt; pure pursuit only (follow the race line)
Between 1.6 m and 1.1 m  -&gt; mix of both
Within 1.1 m             -&gt; FTG takes over completely
Within 0.4 m            -&gt; robot stops
```

Curbs and grass edges show up as obstacles automatically because the TG30 laser is tilted 5 degrees downward (pitch = -0.08726646 radians in the URDF). It detects a 10 cm tall curb from about 1.3 metres ahead. The scan pipeline keeps returns between 2.5 cm and 28 cm above the ground — filters out flat asphalt but keeps curbs and grass.

TF Frames — Why They Matter

map frame — tied to the real world via AMCL. A point at (3.2, 1.5) in map frame means the same physical location every time, every session.

odom frame — starts at zero when the robot boots and drifts over time. Never use odom frame for waypoints.

All waypoints are stored in the map frame. The record_waypoints_live.py script asks TF 'where is base_link in the map frame?' every 20 cm and saves that answer.

```
TF chain at runtime:
map -&gt; odom -&gt; base_footprint -&gt; base_link -&gt; laser_link
  (AMCL)   (EKF: wheel+IMU)
```

Important: Cartographer publishes map->odom during mapping only. AMCL publishes map->odom during localization and racing only. Never run both at the same time — they conflict.

All Commands in One Place

```
# ALWAYS FIRST – every session
# Terminal 1:
roslaunch yahboomcar_nav laser_bringup.launch

# BUILD THE MAP
# Terminal 3:
roslaunch r2_tg30_race mapping_cartographer.launch
# Terminal 4:
roslaunch r2_tg30_race view_mapping.launch
# Drive one full lap. Wait for loop closure. Then Terminal 5:
roslaunch map_server map_saver -f \
    $(rospack find r2_tg30_race)/maps/driveway_course
rosservice call /finish_trajectory 0
rosservice call /write_state \
    "{filename: '$(rospack find r2_tg30_race)/maps/driveway_course.pbstream'}"
# Ctrl-C Terminal 3 and Terminal 4

# START LOCALIZATION
# Terminal 3:
roslaunch r2_tg30_race localization_amcl.launch
# Terminal 4:
roslaunch r2_tg30_race view_localization.launch
# In RViz: click 2D Pose Estimate, click where robot is, drag direction.

# RECORD THE RACE PATH
# Terminal 5:
roslaunch r2_tg30_race record_waypoints_live.py \
    _spacing_m:=0.20 \
    _out_yaml:=$(rospack find r2_tg30_race)/waypoints/waypoints.yaml
# Drive the course at race speed. Ctrl-C when done.

# CLOSE THE LOOP
# Terminal 5:
python3 $(rospack find r2_tg30_race)/scripts/close_loop.py \
    --in_yaml $(rospack find r2_tg30_race)/waypoints/waypoints.yaml \
    --out_yaml $(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml \
    --spacing_m 0.20 --close_thresh 2.0

# PREVIEW THE PATH
# Terminal 5:
roslaunch r2_tg30_race waypoint_map_builder.py \
    _yaml_file:=$(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml

# RACE
# Terminal 6:
roslaunch r2_tg30_race racing_stack.launch \
    waypoint_yaml:=$(rospack find r2_tg30_race)/waypoints/waypoints_closed.yaml
# Terminal 7:
roslaunch r2_tg30_race view_racing.launch
# Hold LB button to race. Let go to stop.
```

Launch Files — What to Run and When

Files you run yourself:

Launch file	Terminal	When to run
yahboomcar_nav/launch/laser_bringup.launch	1	Always first, every session
r2_tg30_race/launch/mapping_cartographer.launch	3	When building the map
r2_tg30_race/launch/view_mapping.launch	4	To watch mapping in RViz
r2_tg30_race/launch/localization_amcl.launch	3	Before recording path and before racing
r2_tg30_race/launch/view_localization.launch	4	To check localization in RViz
r2_tg30_race/launch/racing_stack.launch	6	To start autonomous racing
r2_tg30_race/launch/view_racing.launch	7	To watch racing in RViz

Files started automatically — do not run these yourself:

Launch file	Started by
yahboomcar_bringup/launch/bringup.launch	laser_bringup.launch
yahboomcar_ctrl/launch/yahboom_joy.launch	bringup.launch
r2_tg30_race/launch/scan_cleanup.launch	laser_bringup.launch

Scripts

Script	What it does
record_waypoints_live.py	Saves a waypoint every 20 cm while you drive with the joystick
close_loop.py	Connects the last waypoint back to the first to make a closed loop
waypoint_extractor.py	Alternative — pulls waypoints from a saved rosbag file
waypoint_map_builder.py	Reads the waypoints file and shows the yellow path in RViz
pure_pursuit_twist.py	Steers the robot toward the next waypoint using TF map->base_link
follow_the_gap.py	Watches the laser for obstacles and steers around them
steering_modifier.py	Blends pure pursuit and FTG based on how close obstacles are
twist_to_rosmaster.py	Converts angular.z to linear.y for the R2 motor driver
scan_to_cloud.py	Converts the flat laser scan into a 3D point cloud
cmdvel_gate.py	Only passes autonomous commands through when LB is held
joy_deadman.py	Reads the LB button and tells the system whether auto is on

mode_indicator.py	Shows AUTO or MANUAL text in RViz
bag_control_recorder.py	Starts and stops bag recording using a service call

Configuration Files (all in r2_tg30_race/config/)

pure_pursuit.yaml — how the robot follows the path

```
lookahead_distance: 0.65 # metres ahead to aim for
base_speed:         0.45 # normal driving speed (m/s)
max_speed:          0.70 # fastest allowed (m/s)
min_speed:          0.18 # slowest on tight corners (m/s)
max_angular_z:      1.40 # max steering rate (rad/s)
goal_tolerance:     0.30 # stop within this of the last waypoint (m)
```

follow_the_gap.yaml — obstacle avoidance

```
danger_distance: 1.10 # start reacting at this distance (m)
stop_distance:   0.40 # stop completely if anything this close (m)
front_angle_deg: 120.0 # how wide to look for gaps (degrees)
gap_threshold:   0.70 # minimum gap to steer through (m)
max_speed:       0.40 # top speed during FTG (m/s)
```

pointcloud_to_laserscan.yaml — what counts as an obstacle

```
min_height: 0.025 # skip ground (below 2.5 cm ignored)
max_height: 0.280 # keep curbs and grass up to 28 cm tall
```

joystick_deadman.yaml — LB button setting

```
deadman_button_index: 5 # button 5 = LB on Yahboom controller
```

If LB does not work, run 'rostopic echo /joy' and press LB to find the real button number. Change this value to match.

Troubleshooting

Problem	What to try
Lidar not publishing /scan	Run 'ls /dev/ydlidar' — check the USB cable
Joystick not responding	Run 'ls /dev/input/js0' — plug in before starting
/imu/imu_data not publishing	Restart Terminal 1
Map has doubled lines	Drive slower, complete a full lap, wait longer at start
AMCL particles spread out	Click 2D Pose Estimate in RViz, click where robot is
frame_id: odom in waypoints file	Localization was not running — redo Steps 4 and 5
Gap error in close_loop	Add --close_thresh 4.0 or stop closer to the start
Robot drives but does not steer	Check Terminal 6 (racing_stack.launch) is running
Robot wanders off the path	Drive more slowly. Check particles are tight first.
LB button does nothing	Check deadman_button_index in config/joystick_deadman.yaml
TF lookup errors on startup	Wait for AMCL to converge before starting racing_stack

